Lecture 4: Lifetimes

Practice smart pointers and lifetimes

Willem Vanhulle

DevLab Rust 2025

Tuesday November 25, 2025

github.com/wvhulle/rust-course-ghent

0.1. Educational videos

For the people who are interested in `dyn`, `dyn` is an example of a dynamically-sized-type or `unsized` / `!Sized` type. Read more about such types in https://github.com/pretzelhammer/rust-blog/blob/master/posts/sizedness-in-rust.md

Warning

If forgot to mention that `dyn` trait objects are a kind of type erasure. They erase size, alignment and other compile-time properties of the concrete types.

Youtubers that makes good videos about Rust:

- Jon Gjengset: https://www.youtube.com/c/JonGjengset
- Tris https://www.youtube.com/@NoBoilerplate

Jon Gjengset makes Youtube videos

1.1. Review test

What is a smart pointer?

1.1. Review test

What is a smart pointer?

A pointer with ownership.

Name 10 smart pointers in Rust.

1.1. Review test

What is a smart pointer?

A pointer with ownership.

Name 10 smart pointers in Rust.

`Box<T>`, `Rc<T>`, `Arc<T>`, `RefCell<T>`, `Mutex<T>`, `RwLock<T>`, `Weak<T>`, `Cow<T>`, `String`, `Vec<T>`.

A pointer is generally speaking a type that implements `Deref` trait.

```
1 pub trait Deref {
2     type Target: ?Sized;
3     fn deref(&self) -> &Self::Target;
4 }
```

(playground link)

Which types implement `Deref` but are NOT smart pointers?

1.1. Review test

What is a smart pointer?

A pointer with ownership.

Name 10 smart pointers in Rust.

`Box<T>`, `Rc<T>`, `Arc<T>`, `RefCell<T>`, `Mutex<T>`, `RwLock<T>`, `Weak<T>`, `Cow<T>`, `String`, `Vec<T>`.

A pointer is generally speaking a type that implements `Deref` trait.

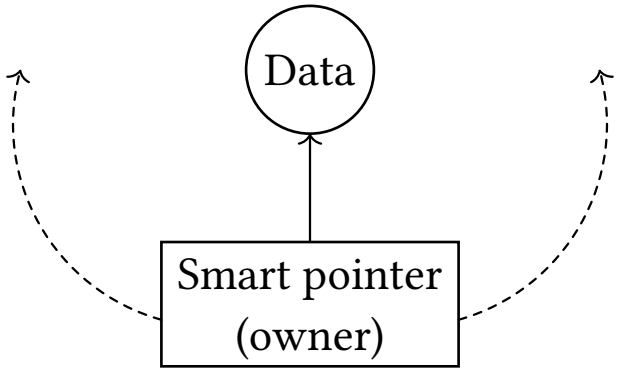
```
1 pub trait Deref {
2     type Target: ?Sized;
3     fn deref(&self) -> &Self::Target;
4 }
```

(playground link)

Which types implement `Deref` but are NOT smart pointers?

References (`&T`, `&mut T`) are primitive pointers without ownership. `Cell<T>` and `MaybeUninit<T>` don't implement `Deref`.

1.2. Visualisation



1.3. Exercises

Solve the exercises in `session-4/examples/` (if you haven't already) in 30 minutes.

Are you done or bored? Implement your own smart pointer. It should own its data and provide shared access to it (single threaded):

- Use `dealloc` and `alloc` for unsafe memory management
- Use `AtomicUsize` for reference counting

Solution in `main.rs` (don't read before trying)

2.1. Borrowing with functions

```rust
fn borrows(x: &i32) {
    dbg!(x);
}

fn main() {
    let mut val = 123;

    // Borrow `val` for the function call.
    borrows(&val);

    // Borrow has ended and we're free to mutate.
    val += 5;
}
```

(playground link)

2.2. Returning Borrows

```rust
fn identity(x: &i32) -> &i32 {
    x
}

fn main() {
    let mut x = 123;
    let out = identity(&x);
    // x = 5; // 🛠️❌ `x` is still borrowed!
    dbg!(out);
}
```

(playground link)

How does the compiler check `out` is not a dangling reference?

2.2. Returning Borrows

```rust
fn identity(x: &i32) -> &i32 {
    x
}

fn main() {
    let mut x = 123;
    let out = identity(&x);
    // x = 5; // 🛠️❌ `x` is still borrowed!
    dbg!(out);
}
```

(playground link)

How does the compiler check `out` is not a dangling reference?

By checking the lifetimes of the references.

Where are the lifetimes?

2.2. Returning Borrows

```rust
fn identity(x: &i32) -> &i32 {
    x
}

fn main() {
    let mut x = 123;
    let out = identity(&x);
    // x = 5; // 🛠️❌ `x` is still borrowed!
    dbg!(out);
}
```

(playground link)

How does the compiler check `out` is not a dangling reference?

By checking the lifetimes of the references.

Where are the lifetimes?

They are hidden in the signature (**lifetime elision**).

2.3. Multiple Borrows

```
1  fn multiple(a: &i32, b: &i32) -> &i32 {
2      todo!("Return either `a` or `b`")
3  }
4
5  fn main() {
6      let mut a = 5; let mut b = 10;
7      let r = multiple(&a, &b);
8
9      // Which one is still borrowed?
10     // Should either mutation be allowed?
11     a += 7; b += 7;
12     dbg!(r);
13 }
```

(playground link)

Why does this code not compile?

2.3. Multiple Borrows

```
1  fn multiple(a: &i32, b: &i32) -> &i32 {
2      todo!("Return either `a` or `b`")
3  }
4
5  fn main() {
6      let mut a = 5; let mut b = 10;
7      let r = multiple(&a, &b);
8
9      // Which one is still borrowed?
10     // Should either mutation be allowed?
11     a += 7; b += 7;
12     dbg!(r);
13 }
```

(playground link)

Why does this code not compile?

Compiler cannot derive which reference to return, so both are considered borrowed.

2.4. Borrow both

```
1   fn pick<'a>(c: bool, a: &'a i32, b: &'a i32) -> &'a i32 {
2       if c { a } else { b }
3   }
4
5   fn main() {
6       let mut a = 5; let mut b = 10;
7       let r = pick(true, &a, &b);
8       // Which one is still borrowed?
9       // Should either mutation be allowed?
10      // a += 7;
11      // b += 7;
12      dbg!(r);
13  }
```

(playground link)

Which argument is borrowed?

2.4. Borrow both

```
1   fn pick<'a>(c: bool, a: &'a i32, b: &'a i32) -> &'a i32 {
2       if c { a } else { b }
3   }
4
5   fn main() {
6       let mut a = 5; let mut b = 10;
7       let r = pick(true, &a, &b);
8       // Which one is still borrowed?
9       // Should either mutation be allowed?
10      // a += 7;
11      // b += 7;
12      dbg!(r);
13  }
```

(playground link)

Which argument is borrowed?

Both `a` and `b` are borrowed for the lifetime of `r`. Compiler looks only at signature.

Don’t borrow from independent owners

2.5. Borrow One

It is possible to borrow from only one argument by introducing a second lifetime. (Even why they are both references)

Open example file for demonstration: `find-nearest.rs`

2.6. Lifetime Elision

```
1   fn only_args(a: &i32, b: &i32) {
2       todo!();
3   }
4
5   fn identity(a: &i32) -> &i32 {
6       a
7   }
8
9   struct Foo(i32);
10  impl Foo {
11      fn get(&self, other: &i32) -> &i32
        {
12          &self.0
13      }
14  }                          (playground link)
```

2.6. Lifetime Elision

```rust
fn only_args(a: &i32, b: &i32) {
    todo!();
}

fn identity(a: &i32) -> &i32 {
    a
}

struct Foo(i32);
impl Foo {
    fn get(&self, other: &i32) -> &i32 {
        &self.0
    }
}
```

(playground link)

2.6.1. Rules for lifetime elision

... for references in argument or return position:

- Each argument which does not have a lifetime annotation is given one.
- If there is only **one argument lifetime**, it is given to all un-annotated return values.
- If there are **multiple argument lifetimes**, but the **first one is for self**, that lifetime is given to all un-annotated return values.

2.6. Lifetime Elision

```rust
fn only_args(a: &i32, b: &i32) {
    todo!();
}

fn identity(a: &i32) -> &i32 {
    a
}

struct Foo(i32);
impl Foo {
    fn get(&self, other: &i32) -> &i32 {
        &self.0
    }
}
```

(playground link)

2.6.1. Rules for lifetime elision

... for references in argument or return position:

- Each argument which does not have a lifetime annotation is given one.
- If there is only **one argument lifetime**, it is given to all un-annotated return values.
- If there are **multiple argument lifetimes**, but the **first one is for self**, that lifetime is given to all un-annotated return values.

Warning

Lifetime elision rules are tricky and very important! Learn them by heart!

2.6. Lifetime Elision

```rust
fn only_args(a: &i32, b: &i32) {
    todo!();
}

fn identity(a: &i32) -> &i32 {
    a
}

struct Foo(i32);
impl Foo {
    fn get(&self, other: &i32) -> &i32 {
        &self.0
    }
}
```

(playground link)

2.6.1. Rules for lifetime elision

... for references in argument or return position:

- Each argument which does not have a lifetime annotation is given one.
- If there is only **one argument lifetime**, it is given to all un-annotated return values.
- If there are **multiple argument lifetimes**, but the **first one is for self**, that lifetime is given to all un-annotated return values.

Warning

Lifetime elision rules are tricky and very important! Learn them by heart!

Complete elided lifetimes in the example code in example file `lifetime-elision.rs`.

2.7. Lifetimes in Data Structures

```rust
#[derive(Debug)]
enum HighlightColor {
    Pink,
    Yellow,
}

#[derive(Debug)]
struct Highlight<'document> {
    slice: &'document str,
    color: HighlightColor,
}

fn main() {
    let doc = String::from("The quick brown fox jumps over the lazy dog.");
    let noun = Highlight { slice: &doc[16..19], color: HighlightColor::Yellow };
    let verb = Highlight { slice: &doc[20..25], color: HighlightColor::Pink };
    // drop(doc);
    dbg!(noun);
    dbg!(verb);
}
```

(playground link)

What does the lifetime `'document` represent?

2.7. Lifetimes in Data Structures

```rust
#[derive(Debug)]
enum HighlightColor {
    Pink,
    Yellow,
}

#[derive(Debug)]
struct Highlight<'document> {
    slice: &'document str,
    color: HighlightColor,
}

fn main() {
    let doc = String::from("The quick brown fox jumps over the lazy dog.");
    let noun = Highlight { slice: &doc[16..19], color: HighlightColor::Yellow };
    let verb = Highlight { slice: &doc[20..25], color: HighlightColor::Pink };
    // drop(doc);
    dbg!(noun);
    dbg!(verb);
}
```

(playground link)

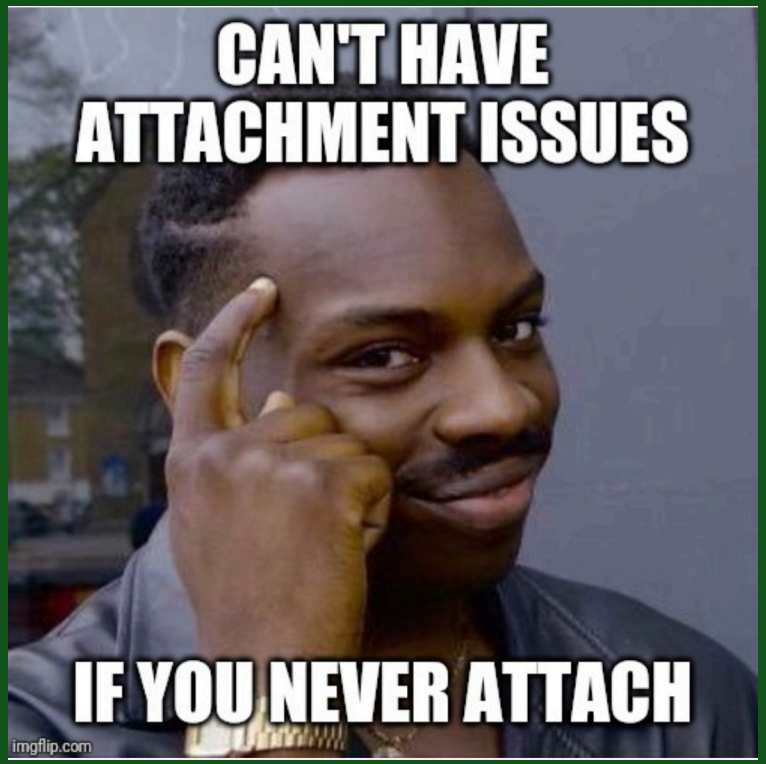
What does the lifetime `'document` represent?

The lifetime of the slice refering to `doc` String in `main()` needs to exceed any instance of `Highlight` (that contains that slice).

2.8. Guidelines

- Types with borrowed data **force users to hold on to the original** data. This can be performant (but harder):
  - creating lightweight views
  - allocation-free parsers
- When possible, make data structures own their data directly.

Don’t develop attachment issues with lifetimes



2.9. Exercises:

- introductory exercises in Rustlings chapter 16: Lifetimes (clone the Rustlings repo)
- advanced exercise in example file `protobuf-parsing.rs`. (in this session's examples folder)

2.10. More reading material

For those interested: read this Blog post by PretzelHamer titled “Common Rust Lifetime Misconceptions”

2.11. Quiz

It is possible to write large Rust programs without ever using lifetimes. True or false?

2.11. Quiz

It is possible to write large Rust programs without ever using lifetimes. True or false?

Not quite true. Lifetimes are almost always omitted because of lifetime elision rules. You need them for implementing iterators.

2.12. Demonstration

```
1  struct ByteIter<'a> {
2      remainder: &'a [u8]
3  }
4
5  impl<'a> ByteIter<'a> {
6      fn next(&mut self) -> Option<&u8> {
7          if self.remainder.is_empty() {
8              None
9          } else {
10             let byte = &self.remainder[0];
11             self.remainder = &self.remainder[1..];
12             Some(byte)
13         }
14     }
15 }
16
17 fn main() {
18     let mut bytes = ByteIter { remainder: b"1" };
19     assert_eq!(Some(&b'1'), bytes.next());
20     assert_eq!(None, bytes.next());
21 }
```

(playground link)

2.12. Demonstration

```rust
fn main() {
    let mut bytes = ByteIter { remainder: b"1123" };
    let byte_1 = bytes.next();
    let byte_2 = bytes.next();
    if byte_1 == byte_2
        // do something
    }
}
```

(playground link)

Will it compile?

2.12. Demonstration

```
1 fn main() {
2     let mut bytes = ByteIter { remainder: b"1123" };
3     let byte_1 = bytes.next();
4     let byte_2 = bytes.next();
5     if byte_1 == byte_2
6         // do something
7     }
8 }
```

(playground link)

Will it compile?

No.

```
1 error[E0499]: cannot borrow `bytes` as mutable more than once at a time
2   --> src/main.rs:20:18
3    |
4 19 |     let byte_1 = bytes.next();
5    |                  ----- first mutable borrow occurs here
6 20 |     let byte_2 = bytes.next();
7    |                  ^^^^^ second mutable borrow occurs here
8 21 |     if byte_1 == byte_2 {
9    |        ------ first borrow later used here
```

(playground link)

2.12. Demonstration

Fill in missing lifetimes and name them properly

```
1  struct ByteIter<'remainder> {
2      remainder: &'remainder [u8]
3  }
4
5  impl<'remainder> ByteIter<'remainder> {
6      fn next<'mut_self>(&'mut_self mut self) -> Option<&'mut_self u8> {
7          if self.remainder.is_empty() {
8              None
9          } else {
10             let byte = &self.remainder[0];
11             self.remainder = &self.remainder[1..];
12             Some(byte)
13         }
14     }
15 }
```

(playground link)

2.13. Lifetime bounds

```
1 struct Wrapper<T> {
2     value: T,
3 }
```
(playground link)

Wherever the type generic `T` appears only owned types may be used. True or false?

2.13. Lifetime bounds

```
1 struct Wrapper<T> {
2     value: T,
3 }
```
(playground link)

Wherever the type generic `T` appears only owned types may be used. True or false?

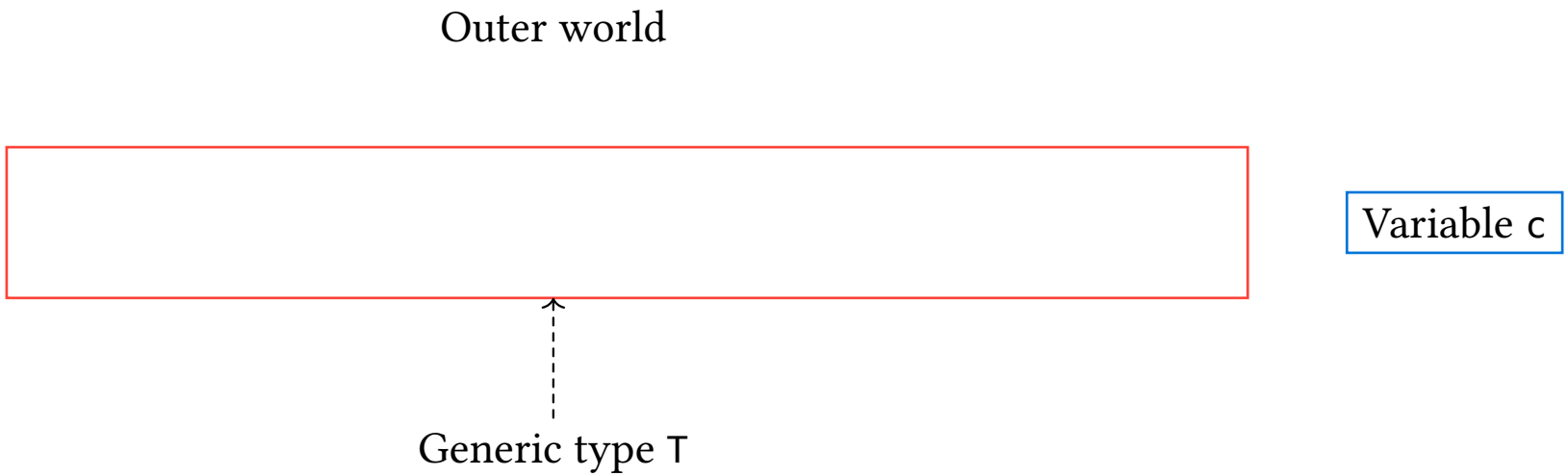
False. `T` may contain references too.

2.13. Lifetime bounds

```
1 struct Wrapper<T> {
2     value: T,
3 }
```
(playground link)

Wherever the type generic `T` appears only owned types may be used. True or false?

False. `T` may contain references too.

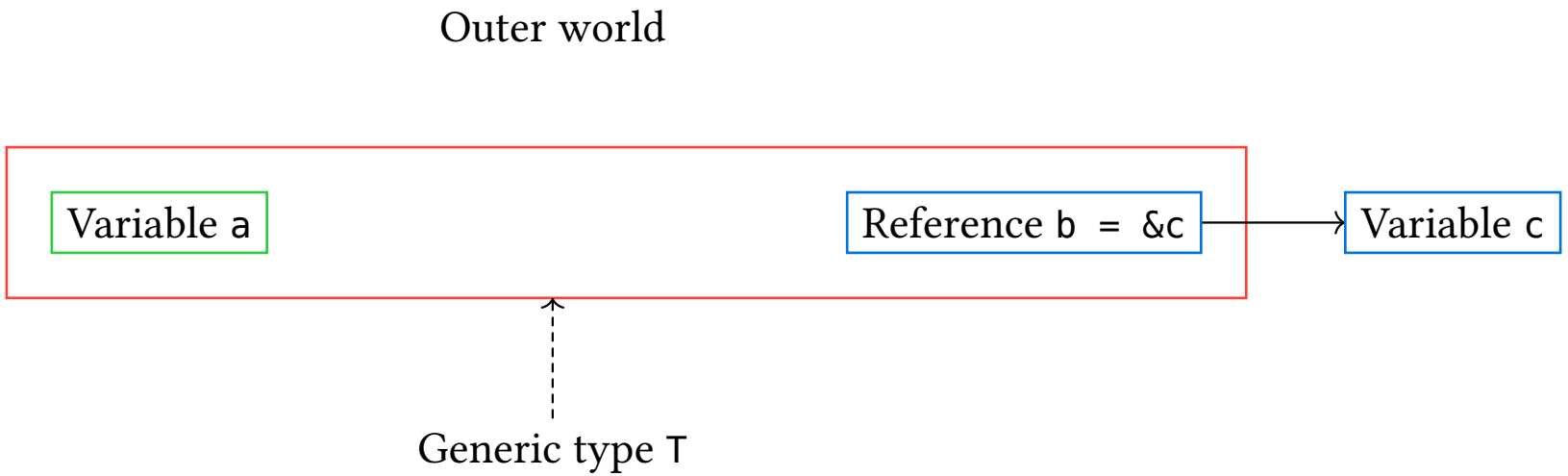


2.13. Lifetime bounds

```
1 struct Wrapper<T> {
2     value: T,
3 }
```
(playground link)

Wherever the type generic `T` appears only owned types may be used. True or false?

False. `T` may contain references too.

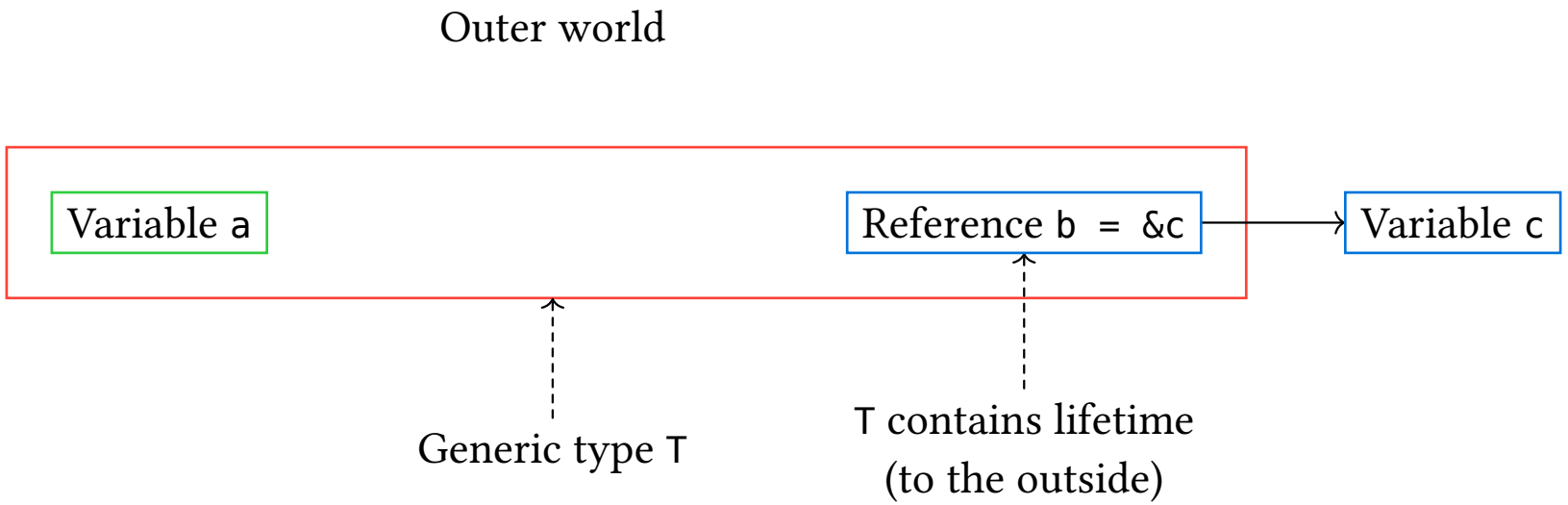


2.13. Lifetime bounds

```
1 struct Wrapper<T> {
2     value: T,
3 }
```
(playground link)

Wherever the type generic `T` appears only owned types may be used. True or false?

False. `T` may contain references too.



In this situation: `T: 'out` where `'out` is the lifetime of `b` (may be implicit in `T`)

2.13. Lifetime bounds

Type compatibility for generic type parameters:

Type Variable	`T`	`&T`	`&mut T`
Examples	`i32`, `&i32`, `&mut i32`, `&&i32`, ...	`&i32`, `&&i32`, `&&mut i32`, ...	`&mut i32`, `&mut &mut i32`, `&mut &i32`, ...

```rust
trait Trait {}
impl<T> Trait for T {}
impl<T> Trait for &T {} // ❌
impl<T> Trait for &mut T {} // ❌
```

(playground link)

Why do the last two impls not compile?

2.13. Lifetime bounds

Type compatibility for generic type parameters:

Type Variable	`T`	`&T`	`&mut T`
Examples	`i32`, `&i32`, `&mut i32`, `&&i32`, ...	`&i32`, `&&i32`, `&&mut i32`, ...	`&mut i32`, `&mut &mut i32`, `&mut &i32`, ...

```
1 trait Trait {}
2 impl<T> Trait for T {}
3 impl<T> Trait for &T {} // ❌
4 impl<T> Trait for &mut T {} // ❌                    (playground link)
```

Why do the last two impls not compile?

The compiler doesn't allow us to define an implementation of `Trait` for `&T` and `&mut T` since it would conflict with the implementation of `Trait` for `T` which already includes all of `&T` and `&mut T`

2.14. `'static` lifetime

The `'static` is a special lifetime (not to be confused with the `'static` keyword for variables). It stands for the longest / maximum lifetime.

If `T: 'static` then T must be valid for the entire program. True or false?

2.14. `'static` lifetime

The `'static` is a special lifetime (not to be confused with the `'static` keyword for variables). It stands for the longest / maximum lifetime.

If `T: 'static` then T must be valid for the entire program. True or false?

False. `T: 'static` means that `T` does **not contain any non-'static references**. It may only contain `'static` references or no references at all.

2.14. `'static` lifetime

The `'static` is a special lifetime (not to be confused with the `'static` keyword for variables). It stands for the longest / maximum lifetime.

If `T: 'static` then T must be valid for the entire program. True or false?

False. `T: 'static` means that `T` does **not contain any non-'static references**. It may only contain `'static` references or no references at all.

Examples: primitive types: `i32`, `f64`, `bool`, `char`.

Conclusion:

- `T: 'static` should be read as *"T can live at least as long as a `'static` lifetime"*

2.14. `'static` lifetime

The `'static` is a special lifetime (not to be confused with the `'static` keyword for variables). It stands for the longest / maximum lifetime.

If `T: 'static` then T must be valid for the entire program. True or false?

False. `T: 'static` means that `T` does **not contain any non-'static references**. It may only contain `'static` references or no references at all.

Examples: primitive types: `i32`, `f64`, `bool`, `char`.

Conclusion:

- `T: 'static` should be read as *"T can live at least as long as a `'static` lifetime"*
- if `T: 'static` then `T` can be a borrowed type with a `'static` lifetime *or* an owned type

2.14. `'static` lifetime

The `'static` is a special lifetime (not to be confused with the `'static` keyword for variables). It stands for the longest / maximum lifetime.

If `T: 'static` then T must be valid for the entire program. True or false?

False. `T: 'static` means that `T` does **not contain any non-'static references**. It may only contain `'static` references or no references at all.

Examples: primitive types: `i32`, `f64`, `bool`, `char`.

Conclusion:

- `T: 'static` should be read as *"T can live at least as long as a `'static` lifetime"*
- if `T: 'static` then `T` can be a borrowed type with a `'static` lifetime *or* an owned type
- since `T: 'static` includes owned types that means `T`
  - can be dynamically allocated at run-time
  - does not have to be valid for the entire program
  - can be safely and freely mutated
  - can be dynamically dropped at run-time
  - can have lifetimes of different durations

`T: 'static` and `impl Trait + 'static`

```
const INFO: &'static str = "I live for the
entire program!";
```

```
static ARRAY: [i32; 3] = [1, 2, 3];
```

2.15. Recent change in compiler

Since 2025 (Rust edition 2024), `impl` blocks in the return type position now capture lifetimes in argument position automatically.

```rust
fn multiply_adapter(
    iter: impl Iterator<Item = i32>,
    factor: &Wrapper,
) -> impl Iterator<Item = i32> {
    iter.map(move |x| x * factor.factor)
}
```

(playground link)

See example `impl-return.rs` for demonstration.

3.1. Motivation

If you want to iterate over the contents of an array, you'll need to define:

- Some state to keep track of where you are in the iteration process, e.g. an index.
- A condition to determine when iteration is done.
- Logic for updating the state of iteration each loop.
- Logic for fetching each element using that iteration state.

```
1 for (int i = 0; i < array_len; i += 1) {
2     int elem = array[i];
3 }                                          (playground link)
```

3.1. Motivation

If you want to iterate over the contents of an array, you'll need to define:

- Some state to keep track of where you are in the iteration process, e.g. an index.
- A condition to determine when iteration is done.
- Logic for updating the state of iteration each loop.
- Logic for fetching each element using that iteration state.

```
1 for (int i = 0; i < array_len; i += 1) {
2     int elem = array[i];
3 }                                          (playground link)
```

What is the closest C equivalent of this to Rust?

3.1. Motivation

If you want to iterate over the contents of an array, you'll need to define:

- Some state to keep track of where you are in the iteration process, e.g. an index.
- A condition to determine when iteration is done.
- Logic for updating the state of iteration each loop.
- Logic for fetching each element using that iteration state.

```
1 for (int i = 0; i < array_len; i += 1) {
2     int elem = array[i];
3 }
```
(playground link)

What is the closest C equivalent of this to Rust?

It uses a pointer that is incremented instead of an index.

```
1 for (int *ptr = array; ptr < array + len; ptr += 1) {
2     int elem = *ptr;
3 }
```
(playground link)

3.2. Iterator trait

```
1  struct SliceIter<'s> {
2      slice: &'s [i32],
3      i: usize,
4  }
5
6  impl<'s> Iterator for SliceIter<'s> {
7      type Item = &'s i32;
8
9      fn next(&mut self) -> Option<Self::Item> {
10         if self.i == self.slice.len() {
11             None
12         } else {
13             let next = &self.slice[self.i];
14             self.i += 1;
15             Some(next)
16         }
17     }
18 }
19                                    (playground link)
```

The Iterator trait defines how an object can be used to **produce a sequence of values**.

```
1     fn main() {
2         let slice = &[2, 4, 6, 8];
3         let iter = SliceIter { slice, i:
          0 };
4         for elem in iter {
5             dbg!(elem);
6         }
7  }                                  (playground link)
```

3.2. Iterator trait

Why are Rust iterators lazy?

3.2. Iterator trait

Why are Rust iterators lazy?

You can call `next()` to get the next element only when you need it.

What happens when you call `next()` after the iterator returned `None`

3.2. Iterator trait

Why are Rust iterators lazy?

You can call `next()` to get the next element only when you need it.

What happens when you call `next()` after the iterator returned `None`

We don't know. Only fused iterators guarantee `None` forever after.

```
1 fn main() {
2     let mut iter = SliceIter { slice: &[1, 2], i: 0 };
3     assert_eq!(Some(&1), iter.next());
4     assert_eq!(Some(&2), iter.next());
5     assert_eq!(None, iter.next());
6     assert_eq!(??? , iter.next()); // What happens here?
7 }
```

(playground link)

3.3. Generators

Give an example of an infinite iterator.

3.3. Generators

Give an example of an infinite iterator.

The range `0..` is an infinite iterator of integers starting from 0.

Iterators can also be generated with coroutines (called generators in Rust):

```rust
fn counter() -> impl Iterator<Item = i32> {
    gen {
        let mut count = 0;
        loop {
            yield count;
            count += 1;
        }
    }
}
```

(playground link)

(Also an infinite iterator)

3.4. Helpers

the Iterator trait provides **helper methods** that can be used to transform iterators:

```rust
fn main() {
    let result: i32 = (1..=10) // Create a range from 1 to 10
        .filter(|x| x % 2 == 0) // Keep only even numbers
        .map(|x| x * x) // Square each number
        .sum(); // Sum up all the squared numbers

    println!("The sum of squares of even numbers from 1 to 10 is: {}", result);
}
```

(playground link)

What is another name for iterator helpers?

3.4. Helpers

the Iterator trait provides **helper methods** that can be used to transform iterators:

```rust
fn main() {
    let result: i32 = (1..=10) // Create a range from 1 to 10
        .filter(|x| x % 2 == 0) // Keep only even numbers
        .map(|x| x * x) // Square each number
        .sum(); // Sum up all the squared numbers

    println!("The sum of squares of even numbers from 1 to 10 is: {}", result);
}
```

(playground link)

What is another name for iterator helpers?

Adapters (or operators or combinators).

How many adapters are there in the standard library?

3.4. Helpers

the Iterator trait provides **helper methods** that can be used to transform iterators:

```rust
fn main() {
    let result: i32 = (1..=10) // Create a range from 1 to 10
        .filter(|x| x % 2 == 0) // Keep only even numbers
        .map(|x| x * x) // Square each number
        .sum(); // Sum up all the squared numbers

    println!("The sum of squares of even numbers from 1 to 10 is: {}", result);
}
```

(playground link)

What is another name for iterator helpers?

Adapters (or operators or combinators).

How many adapters are there in the standard library?

Approximately 40.

3.5. Collect

Build collections from iterators:

```rust
fn main() {
    let primes = vec![2, 3, 5, 7];
    let prime_squares = primes.into_iter().map(|p| p * p).collect::<Vec<_>>();
    println!("prime_squares: {prime_squares:?}");
}
```

(playground link)

What are the collections that we can collect into?

3.5. Collect

Build collections from iterators:

```
1 fn main() {
2     let primes = vec![2, 3, 5, 7];
3     let prime_squares = primes.into_iter().map(|p| p * p).collect::<Vec<_>>();
4     println!("prime_squares: {prime_squares:?}");
5 }
```

(playground link)

What are the collections that we can collect into?

Vectors, HashMaps, HashSets, BtreeMap, BtreeSet, VecDeque, ... (your own collections too!)

Target collection type is either implicit or specified with:

3.5. Collect

Build collections from iterators:

```rust
fn main() {
    let primes = vec![2, 3, 5, 7];
    let prime_squares = primes.into_iter().map(|p| p * p).collect::<Vec<_>>();
    println!("prime_squares: {prime_squares:?}");
}
```

(playground link)

What are the collections that we can collect into?

Vectors, HashMaps, HashSets, BtreeMap, BtreeSet, VecDeque, ... (your own collections too!)

Target collection type is either implicit or specified with:

- Turbofishing: `collect::<Vec<_>>()`
- Type annotation: `let v: Vec<_> = iterator.collect();`

3.6. Consuming adapters

Consume the iterator and produce a final value:

```
let nums = vec![1, 2, 3, 4, 5];
nums.iter().collect::<Vec<_>>();  // collect into collection
nums.iter().sum::<i32>();          // reduce to single value
nums.iter().count();               // count elements
nums.iter().fold(0, |acc, x| acc + x); // general reduction
nums.iter().for_each(|x| println!("{}", x)); // side effects          (playground link)
```

Other examples: `max`, `min`, `find`, `any`, `all`, `partition`, `reduce`.

Why are they called “consuming”?

3.6. Consuming adapters

Consume the iterator and produce a final value:

```rust
let nums = vec![1, 2, 3, 4, 5];
nums.iter().collect::<Vec<_>>();  // collect into collection
nums.iter().sum::<i32>();         // reduce to single value
nums.iter().count();              // count elements
nums.iter().fold(0, |acc, x| acc + x); // general reduction
nums.iter().for_each(|x| println!("{}", x)); // side effects
```

(playground link)

Other examples: `max`, `min`, `find`, `any`, `all`, `partition`, `reduce`.

Why are they called “consuming”?

They call `next()` until `None`.

3.7. Nonconsuming adapters

Transform an iterator into another iterator (lazy):

```
let nums = vec![1, 2, 3, 4, 5];
nums.iter().map(|x| x * 2);         // transform elements
nums.iter().filter(|x| *x % 2 == 0); // keep matching elements
nums.iter().take(3);                // limit to first N
nums.iter().skip(2);                // skip first N
nums.iter().enumerate();            // add indices
nums.iter().chain([6, 7].iter());   // concatenate
nums.iter().zip(['a', 'b'].iter()); // pair up                    (playground link)
```

Other examples: `flatten`, `flat_map`, `peekable`, `rev`, `cycle`, `cloned`.

Why are they called “nonconsuming”?

3.7. Nonconsuming adapters

Transform an iterator into another iterator (lazy):

```
let nums = vec![1, 2, 3, 4, 5];
nums.iter().map(|x| x * 2);         // transform elements
nums.iter().filter(|x| *x % 2 == 0); // keep matching elements
nums.iter().take(3);                // limit to first N
nums.iter().skip(2);                // skip first N
nums.iter().enumerate();            // add indices
nums.iter().chain([6, 7].iter());   // concatenate
nums.iter().zip(['a', 'b'].iter()); // pair up                    (playground link)
```

Other examples: `flatten`, `flat_map`, `peekable`, `rev`, `cycle`, `cloned`.

Why are they called “nonconsuming”?

They don’t run until a consuming adapter is called.

3.8. Overhead

What is the cost of using lots of iterator adapters?

3.8. Overhead

What is the cost of using lots of iterator adapters?

There is no runtime overhead because most adapters work by reference and can be inlined or optimized by the compiler.

Exercises: Rustlings, chapter 18: Iterators

3.9. `AsyncIterator` adapters

Fragment from my EuroRust 2025 talk on functional async programming in Rust (github.com/wvhulle/streams-eurorust-2025/):

```rust
let results: Vec<String> = tcp_stream
    .filter_map(|conn| ready(conn.ok()))
    .filter(|stream| ready(should_process(stream)))
    .then(|stream| process_stream(stream))
    .filter_map(|result| ready(result.ok()))
    .filter(|msg| ready(msg.len() > 10))
    .take(5)
    .collect()
    .await;
```

(playground link)

Benefits:

- Each operation is isolated
- Testable
- Reusable

3.10. IntoIterator

IntoIterator defines how to create an iterator for a type:

```
1   struct Grid {
2       x_coords: Vec<u32>,
3       y_coords: Vec<u32>,
4   }
5
6   impl IntoIterator for Grid {
7       type Item = (u32, u32);
8       type IntoIter = GridIter;
9       fn into_iter(self) -> GridIter {
10          GridIter { grid: self, i: 0, j: 0 }
11      }
12  }
```

(playground link)

3.11. The “actual” / physical iterator

```rust
struct GridIter {
    grid: Grid,
    i: usize,
    j: usize,
}

impl Iterator for GridIter {
    type Item = (u32, u32);

    fn next(&mut self) -> Option<(u32, u32)> {
        if self.i >= self.grid.x_coords.len() {
            self.i = 0;
            self.j += 1;
            if self.j >= self.grid.y_coords.len() {
                return None;
            }
        }
        let res = Some((self.grid.x_coords[self.i], self.grid.y_coords[self.j]));
        self.i += 1;
        res
    }
}
```

(playground link)

3.12. Different `IntoIterator` implementations

Iterating over a vector like this will consume it:

```
let v = vec![1, 2, 3];
for elem in v {
    dbg!(elem);
}
// v is no longer usable here
```

(playground link)

Iterating over a reference to a vector will not consume it:

```
let v = vec![1, 2, 3];
for elem in &v {
    dbg!(elem);
}
// v is still usable here
```

(playground link)

3.13. Iterate over Grid by reference

You might have multiple ways to create an iterator for a type.

Usually, by value (consuming) or by reference (non-consuming):

```
impl<'a> IntoIterator for &'a Grid {
    type Item = (u32, u32);
    type IntoIter = GridRefIter<'a>;
    fn into_iter(self) -> GridRefIter<'a> {
        GridRefIter { grid: self, i: 0, j: 0 }
    }
}
```

(playground link)

From a mutable reference (non-consuming + mutable):

```
impl<'a> IntoIterator for &'a mut Grid {
    type Item = (u32, u32);
    type IntoIter = GridMutRefIter<'a>;
    fn into_iter(self) -> GridMutRefIter<'a> {
        GridMutRefIter { grid: self, i: 0, j: 0 }
    }
}
```

(playground link)

See example file `into-iterator-ref.rs`.

3.14. Exercises

Solve the exercise in `tests/iterator-method-chaining.rs`

Create your own iterator by implementing `Iterator` and `IntoIterator` in `examples/custom-iterator.rs`

Build your own `Iterator` adapters in `examples/iterator-adapters.rs`